Universidade Estadual de Santa Cruz (UESC)

Etapa 2 — Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Implementação paralela em memória distribuída com MPI.

Relatório acadêmico — DEC107 — Processamento Paralelo

Autores: Enzo Fernandes Sarmento, Jéssica Meira Costa

Disciplina: DEC107 — Processamento Paralelo

Data: 29/10/2026

1. Introdução

O processamento paralelo tem como objetivo utilizar simultaneamente múltiplos recursos computacionais para reduzir o tempo necessário para executar determinadas aplicações. Nesse contexto, busca-se identificar problemas que podem ser divididos em tarefas independentes e analisar como diferentes estratégias de paralelização influenciam o desempenho. Entre os modelos de programação paralela, destaca-se o MPI (*Message Passing Interface*), que permite a execução cooperativa de múltiplos processos por meio da troca de mensagens em ambientes de memória distribuída.

O objetivo desta segunda etapa é implementar e avaliar uma versão paralela do algoritmo de geração do Conjunto de Mandelbrot utilizando MPI, com foco na distribuição do trabalho entre processos em um ambiente de memória distribuída. Para isso, são avaliadas diferentes quantidades de processos e duas estratégias de distribuição do domínio da imagem, considerando métricas como tempo de execução, *speedup*, eficiência, balanceamento de carga e custo de comunicação. Busca-se compreender como a organização do trabalho e a comunicação entre processos influenciam o desempenho da aplicação.

O Conjunto de Mandelbrot é obtido a partir do comportamento de uma sequência de números complexos. Para cada ponto (c) do plano complexo, considera-se inicialmente ($z_0 = 0$) e aplica-se repetidamente a expressão ($z_{n+1} = z_n^2 + c$). O método *escape-time* determina quantas iterações são necessárias para que o módulo de (z) ultrapasse um determinado raio de escape. Neste projeto, utiliza-se o critério ($|z|^2 > 4$), equivalente a (|z|>2), evitando o cálculo de uma raiz quadrada a cada iteração. O número de iterações realizadas, limitado pelo valor de MAX_ITER, corresponde ao valor armazenado para cada pixel da imagem.

Embora o cálculo de cada pixel possa ser realizado de forma independente, seu custo computacional não é necessariamente uniforme. Pontos localizados fora do conjunto geralmente ultrapassam rapidamente o raio de escape, exigindo poucas iterações, enquanto pontos pertencentes ao conjunto ou próximos à sua fronteira podem exigir muitas iterações. Essa heterogeneidade pode provocar desbalanceamento de carga entre os processos, principalmente quando regiões com custos computacionais distintos são atribuídas aos mesmos processos. Dessa forma, a estratégia de decomposição do domínio da imagem é determinante para distribuir o trabalho de maneira equilibrada e reduzir o tempo durante o qual alguns processos permanecem ociosos aguardando os demais.

O MPI permite que múltiplos processos executem suas tarefas em espaços de memória próprios, comunicando-se explicitamente por meio da troca de mensagens. Neste projeto, a imagem é dividida em faixas horizontais, de modo que cada processo seja responsável pelo cálculo de um subconjunto de linhas. A distribuição estática consiste em definir previamente as tarefas de cada processo, enquanto a distribuição dinâmica atribui novas tarefas durante a execução, conforme os processos ficam disponíveis. A implementação desenvolvida utiliza duas estratégias estáticas: *block*, que distribui as linhas em blocos contíguos, e *cyclic*, que distribui as linhas de maneira intercalada entre os processos. Ao final do processamento, os resultados parciais são reunidos no processo raiz para reconstruir a imagem completa. Na estratégia cíclica, é necessário também reorganizar as linhas recebidas para recuperar a ordem original da imagem.

A análise de desempenho em sistemas distribuídos é importante porque o aumento do número de processos não garante uma redução proporcional no tempo de execução. Além do custo computacional, o desempenho depende do balanceamento da carga, da

sincronização e da comunicação necessária para reunir os resultados. Nesse contexto, a implementação mede o tempo da etapa de coleta dos dados e, na distribuição cíclica, o custo adicional de reorganização das linhas. Essas operações podem reduzir os ganhos obtidos com o paralelismo, especialmente quando a sobrecarga de comunicação se torna significativa em relação ao tempo de processamento. Assim, a avaliação das métricas de tempo de execução, *speedup*, eficiência, balanceamento de carga e custo de comunicação permite compreender os benefícios e as limitações das estratégias utilizadas na geração do Conjunto de Mandelbrot.

2. Metodologia

2.1 Implementação paralela com MPI

A implementação paralela do conjunto de Mandelbrot foi desenvolvida em linguagem C utilizando a biblioteca MPI, que permite a execução do programa em múltiplos processos. A função `MPI_Init` inicializa o ambiente MPI, enquanto `MPI_Comm_rank` identifica o rank de cada processo e `MPI_Comm_size` determina o número total de processos utilizados.

A função `MPI_Bcast` é utilizada para transmitir as configurações da execução do processo raiz (rank 0) para os demais processos, garantindo que todos utilizem os mesmos parâmetros. O trabalho é distribuído entre os processos por meio de duas estratégias: *block*, na qual cada processo calcula um conjunto de linhas consecutivas da imagem, e *cyclic*, na qual as linhas são distribuídas de forma intercalada.

Para reunir os resultados, são utilizadas as funções `MPI_Gather` e `MPI_Gatherv`. A primeira é empregada quando os processos possuem a mesma quantidade de dados, enquanto a segunda permite coletar quantidades diferentes. Na distribuição *cyclic*, `MPI_Gatherv` reúne os resultados em um buffer temporário, e o processo raiz reorganiza as linhas para reconstruir a imagem corretamente.

A função `MPI_Reduce` é utilizada para calcular os tempos máximo, mínimo e a soma dos tempos de computação local, permitindo obter informações sobre o desempenho e o balanceamento da carga. Também são utilizadas chamadas de `MPI_Gather` para reunir os tempos de computação e as quantidades de pixels processados por cada processo.

A sincronização é realizada com `MPI_Barrier`, que estabelece pontos de coordenação entre os processos antes do cálculo, antes da coleta dos resultados e ao final da etapa paralela. A função `MPI_Wtime` mede os tempos de execução, incluindo o tempo de computação, o tempo de coleta dos resultados e, na estratégia *cyclic*, o tempo de reorganização das linhas. A gravação dos arquivos de imagem é cronometrada separadamente.

Não foi necessário utilizar diretamente `MPI_Send` e `MPI_Recv`, pois a comunicação entre os processos é realizada principalmente por operações coletivas, adequadas à distribuição das configurações e à reunião dos resultados. Ao final, o processo raiz grava as imagens nos arquivos `mandelbrot_mpi.bin` e `mandelbrot_mpi.ppm`, e todos os processos encerram a execução por meio de `MPI_Finalize`.

2.2 Estratégias de distribuição

Foram implementadas duas estratégias de distribuição estática do trabalho: **block** e **cyclic**.

Na estratégia **block**, as linhas da imagem são divididas em blocos de linhas consecutivas entre os processos. A quantidade de linhas é calculada com base na altura da imagem e no número de processos. Quando a divisão não é exata, os primeiros processos recebem uma linha adicional. Dessa forma, cada processo calcula os pixels das linhas que lhe foram atribuídas.

Na estratégia **cyclic**, as linhas são distribuídas de forma intercalada entre os processos. Por exemplo, com quatro processos, o processo 0 calcula as linhas 0, 4 e 8; o processo 1 calcula as linhas 1, 5 e 9; o processo 2 calcula as linhas 2, 6 e 10; e o processo 3 calcula as linhas 3, 7 e 11. Essa distribuição continua até que todas as linhas da imagem sejam atribuídas.

Nas duas estratégias, a granularidade adotada é a linha da imagem. Cada processo calcula todos os pixels das linhas que recebe, e a quantidade de linhas atribuída depende da altura da imagem, do número de processos e da estratégia selecionada.

O processo de rank 0 é responsável por distribuir as configurações da execução, reunir os resultados calculados pelos processos e reconstruir e salvar a imagem final. Os demais processos calculam suas linhas atribuídas e participam das operações coletivas de comunicação. Na estratégia *cyclic*, o processo de rank 0 também reorganiza os resultados recebidos para reconstruir a imagem na ordem correta.

As duas estratégias permitem comparar diferentes formas de distribuição estática das linhas e analisar suas influências sobre o tempo de execução e o desempenho do programa paralelo.

2.3 Coleta dos resultados e escrita de saída

A coleta dos resultados é realizada no processo de rank 0 por meio das funções coletivas `MPI_Gather` e `MPI_Gatherv`. Na distribuição por blocos (*block*), utiliza-se `MPI_Gather` quando a altura da imagem é divisível pelo número de processos, de modo que todos recebem a mesma quantidade de linhas. Quando a divisão não é exata, utiliza-se `MPI_Gatherv`, que permite especificar a quantidade de dados recebidos de cada processo e seus respectivos deslocamentos na matriz global.

Na distribuição cíclica (*cyclic*), utiliza-se `MPI_Gatherv` para reunir os resultados em um buffer auxiliar. Como cada processo calcula linhas intercaladas, o processo de rank 0 utiliza a função `memcpy` para copiar cada linha para sua posição correta na matriz global, reconstruindo a imagem na ordem original.

Após a coleta e, quando necessário, a reorganização dos resultados, o processo de rank 0 grava a imagem em dois arquivos: `mandelbrot_mpi.bin`, que armazena os dados da imagem em formato binário, e `mandelbrot_mpi.ppm`, que armazena a imagem em formato PPM, permitindo sua visualização. O tempo de escrita dos arquivos é medido separadamente e não está incluído no intervalo utilizado para medir o tempo de execução paralela.

Dessa forma, os processos MPI participam do cálculo dos pixels e do envio coletivo dos resultados, enquanto o processo de rank 0 reúne os dados, reconstrói a imagem completa e grava os arquivos de saída.

2.4 Hardware e ambiente de execução

Os experimentos foram realizados em um ambiente Linux, utilizando a biblioteca MPI para execução paralela do conjunto de Mandelbrot. As características do hardware e do software utilizado são apresentadas a seguir.

- Sistema operacional: Ubuntu 24.04.1;
- Kernel: Linux 6.14.0-37-generic;
- Arquitetura: x86_64;
- Processador: AMD Ryzen 5 5600H with Radeon Graphics;

- Núcleos físicos: 6;
- Threads lógicas: 12;
- Threads por núcleo: 2;
- Frequência máxima: 4,28 GHz;
- Cache L3: 16 MiB;
- Cache L2: 3 MiB;
- Cache L1d: 192 KiB;
- Cache L1i: 192 KiB;
- NUMA: 1 nó;
- Compilador: GCC 13.3.0.
- Opções de compilação: `-Wall -Wextra -O3 -march=native -ffast-math`.
- Biblioteca matemática: `-lm`.
- Memória RAM: 24GB DDR4
- Implementação MPI e versão: MPICH 4.3.2, gerenciador Hydra;
- Número de processos MPI avaliados: de 1 a 12.
- Estratégias de distribuição: `block` e `cyclic`.
- Comando de execução: `mpirun -np N`, em que N representa o número de processos MPI.
- Binding dos processos: utilizou-se a política padrão do MPICH/Hydra.

Os testes foram realizados variando o número de processos MPI e a estratégia de distribuição, permitindo comparar o desempenho das estratégias implementadas no ambiente descrito.

2.5 Procedimentos de medição de tempo

A medição de desempenho foi realizada utilizando as funções `MPI_Wtime()` e `omp_get_wtime()` nas versões paralela MPI e sequencial, respectivamente. Na versão MPI, foi utilizada a chamada `MPI_Barrier(MPI_COMM_WORLD)` antes do

início da medição principal, garantindo que os processos estivessem sincronizados antes da execução.

O tempo total de geração da imagem na versão MPI foi medido desde o início da região principal de execução até a conclusão das etapas de computação, coleta dos resultados e reorganização das linhas, quando necessária. Para representar a duração total da execução paralela, foi considerado o maior tempo registrado entre os processos. Esse tempo não inclui a escrita dos arquivos em disco, que foi medida separadamente.

O tempo de computação foi medido individualmente em cada processo MPI. A partir dessas medições, foram obtidos os tempos máximo, mínimo e médio de computação, utilizados para analisar a distribuição da carga de trabalho entre os processos.

O custo associado à comunicação e à coleta dos resultados foi avaliado por meio da medição das operações `MPI_Gather` e `MPI_Gatherv`. Na distribuição cíclica (*cyclic*), também foi medido separadamente o tempo de reorganização das linhas da imagem no processo raiz. Essas medidas permitem analisar os custos adicionais envolvidos na consolidação dos resultados, embora o tempo das operações coletivas também possa incluir custos de sincronização e processamento interno da biblioteca MPI.

A escrita dos arquivos de saída nos formatos `.bin` e `.ppm` foi cronometrada separadamente, ficando fora da medição principal do tempo de geração da imagem. Para cada configuração experimental, foram realizadas cinco repetições, permitindo comparar as estratégias de distribuição de linhas por blocos contíguos (*block*) e distribuição cíclica (*cyclic*).

2.6 Métricas utilizadas para avaliação

Para avaliar o desempenho da implementação paralela do conjunto de Mandelbrot utilizando MPI, foram consideradas as métricas de tempo de execução, speedup, eficiência, fator de balanceamento de carga e escalabilidade forte e fraca.

Tempo de execução: representa o tempo necessário para executar o processamento do conjunto de Mandelbrot. Nos experimentos, foram registrados os tempos para diferentes quantidades de processos MPI, variando de 1 a 12, e para as estratégias de distribuição do trabalho *block* e *cyclic*. O tempo principal da execução paralela foi analisado separadamente do tempo de escrita dos arquivos de saída, permitindo avaliar o desempenho do processamento sem incluir o custo da gravação dos resultados.

Speedup: mede quantas vezes a execução paralela foi mais rápida em comparação com a execução sequencial. É calculado pela divisão do tempo sequencial pelo tempo paralelo, conforme a fórmula:

$S(p) = T1 / Tp$

Em que T1 representa o tempo de execução sequencial e Tp representa o tempo de execução paralela com p processos. Quanto maior o speedup, maior o ganho de desempenho obtido com a paralelização.

Eficiência: indica o aproveitamento dos processos utilizados na execução paralela. É calculada pela divisão do speedup pela quantidade de processos:

$E(p) = S(p) / p$

Para expressar a eficiência em porcentagem, o resultado deve ser multiplicado por 100. Valores próximos de 100% indicam um melhor aproveitamento dos processos, enquanto valores menores podem indicar a influência dos custos de comunicação, sincronização e distribuição do trabalho.

Fator de balanceamento de carga: avalia a diferença entre os tempos de processamento dos processos MPI. Neste trabalho, considerando os tempos máximo e mínimo de computação, o fator é calculado pela fórmula:

$FBC = (Tmax - Tmin) / Tmax$

Em que Tmax representa o maior tempo de computação e Tmin representa o menor tempo de computação entre os processos. Valores próximos de zero indicam tempos de processamento semelhantes, enquanto valores maiores indicam diferenças mais acentuadas entre os processos. Quando existe apenas um processo, o fator é zero, pois os tempos máximo e mínimo são iguais.

Escalabilidade forte: avalia o comportamento da aplicação quando a quantidade de processos aumenta, mantendo constante o tamanho do problema. Nos experimentos, essa análise é realizada comparando os tempos de execução, o speedup e a eficiência para diferentes quantidades de processos MPI, mantendo fixa a configuração da imagem. Os modos *padrão* e *Seahorse* são analisados separadamente, pois representam configurações distintas do conjunto de Mandelbrot.

Escalabilidade fraca: avalia o comportamento da aplicação quando a quantidade de processos e o tamanho do problema aumentam proporcionalmente, mantendo aproximadamente constante a quantidade de trabalho por processo. Os experimentos

realizados com diferentes quantidades de processos, mas mantendo o tamanho da imagem fixo em cada configuração, permitem avaliar a escalabilidade forte. Para avaliar a escalabilidade fraca adequadamente, são necessários experimentos adicionais com diferentes tamanhos de imagem, ajustados proporcionalmente à quantidade de processos utilizada.

Essas métricas permitem comparar as estratégias *block* e *cyclic*, avaliar os ganhos de desempenho obtidos com a paralelização e analisar a influência da distribuição do trabalho, da comunicação e do aumento do número de processos sobre o desempenho da aplicação.

2.7 Fator de Balanceamento de Carga

O Fator de Balanceamento de Carga (FBC) foi calculado conforme a definição adotada em sala de aula:

FBC = (T_max - T_min) / T_max

Nessa fórmula, T_max e T_min representam, respectivamente, o maior e o menor tempo de computação entre os processos MPI. Esses tempos foram medidos individualmente em cada processo utilizando a função `MPI_Wtime()`, considerando o intervalo entre o início e o término do cálculo dos pixels atribuídos a cada processo. Em seguida, os valores máximo e mínimo foram obtidos por meio da operação coletiva `MPI_Reduce()`.

A métrica considera apenas o tempo de computação local, sem incluir diretamente os tempos de coleta dos resultados (`MPI_Gather` ou `MPI_Gatherv`) e de reorganização das linhas da imagem. De acordo com essa definição, FBC igual a zero indica balanceamento perfeito, enquanto valores próximos de um indicam forte desbalanceamento. Essa métrica permite comparar como as estratégias *block* e *cyclic* distribuem o trabalho computacional entre os processos MPI.

3. Resultados

3.1 Verificação da corretude

A primeira etapa da avaliação consistiu em verificar se a implementação com MPI preserva os resultados produzidos pela versão serial de referência. Para isso, foram comparadas as matrizes de contagens de iteração geradas pelas duas

implementações, considerando o input padrão e o cenário do Vale dos Cavalos-Marinhos, para as estratégias de distribuição block e cyclic.

O critério adotado foi a igualdade exata das matrizes. A Tabela 1 apresenta os resultados dessa verificação, realizada com quatro processos MPI em cada configuração.

Tabela 1 — Verificação da corretude da implementação MPI

Cenário	Estratégia	Processos	Pixels divergentes	Divergência (%)	Diferença máxima (iterações)	Igualdade exata
Padrão	block	4	0	0,0000%	0	Sim
Padrão	cyclic	4	0	0,0000%	0	Sim
Seahorse	block	4	0	0,0000%	0	Sim
Seahorse	cyclic	4	0	0,0000%	0	Sim

No input padrão, correspondente à imagem de (4096x4096) pixels e MAX_ITER = 1000, as duas estratégias produziram matrizes idênticas à saída serial. O mesmo resultado foi observado no cenário do Vale dos Cavalos-Marinhos, que utiliza a mesma resolução e MAX_ITER = 5000.

Em ambos os cenários, foram comparados 16.777.216 pixels, sem divergências nos valores armazenados. Os hashes SHA-256 dos arquivos binários também coincidiram entre as versões serial e MPI para cada cenário.

Esses resultados demonstram que a decomposição do domínio e a reunião dos resultados não alteraram os valores calculados pelo algoritmo. Portanto, as diferenças de desempenho analisadas nas seções seguintes decorrem da organização da execução paralela, e não de alterações observadas no resultado numérico.

3.2 Desempenho no input padrão

Os experimentos do input padrão foram realizados com resolução de (4096x4096) pixels e MAX_ITER = 1000, variando o número de processos MPI entre 1 e 12. Para cada quantidade de processos, foram avaliadas as estratégias block, que distribui linhas contíguas, e cyclic, que distribui linhas de forma intercalada.

Foram realizadas cinco execuções independentes para cada configuração. O tempo de referência (T_1), registrado no arquivo de benchmark para a execução MPI de um processo, apresentou média de **0,846998 s**. O tempo paralelo (T_p) utilizado na avaliação inclui o intervalo medido desde o início comum da execução até a conclusão do processamento, da coleta dos resultados e, quando aplicável, do rearranjo das linhas. A gravação dos arquivos de saída não integra essa medição.

A Tabela 2 reúne os tempos médios, o *speedup*, a eficiência e o FBC das duas estratégias para todas as quantidades de processos avaliadas.

Tabela 2 — Desempenho MPI no input padrão

Processos	(T_p) block (s)	Speedup	Eficiência	FBC	(T_p) cyclic (s)	Speedup	Eficiência	FBC
1	0,8492	0,997	99,74%	0,00%	0,8930	0,949	94,86%	0,00%
2	0,4513	1,878	93,88%	0,81%	0,4866	1,741	87,04%	0,49%
3	0,3094	2,738	91,27%	0,39%	0,3521	2,406	80,19%	0,52%
4	0,3517	2,410	60,24%	66,54%	0,2873	2,949	73,73%	0,95%
5	0,3503	2,419	48,38%	92,29%	0,2547	3,332	66,64%	2,74%
6	0,3159	2,685	44,76%	93,15%	0,2256	3,755	62,59%	1,69%
7	0,2251	3,768	53,82%	91,86%	0,2319	3,657	52,24%	13,18%
8	0,2642	3,208	40,10%	92,27%	0,2146	3,956	49,45%	12,97%
9	0,3209	2,641	29,35%	94,21%	0,2135	3,977	44,19%	12,91%
10	0,2688	3,158	31,58%	94,07%	0,2193	3,867	38,67%	13,85%
11	0,2761	3,164	28,76%	91,95%	0,2349	3,625	32,95%	21,41%
12	0,2911	2,921	24,34%	92,29%	0,2595	3,346	27,88%	32,37%

Os resultados mostram que a distribuição cyclic apresentou o menor tempo médio na maior parte das configurações com quatro ou mais processos, embora não tenha sido a melhor opção em todos os casos. O menor tempo médio observado foi de **0,213505 s**,

utilizando nove processos e a estratégia cyclic, com *speedup* médio de 3,977 e eficiência média de 44,19%.

Esse resultado indica que a distribuição intercalada permitiu reduzir o tempo de execução em relação às faixas contíguas nas configurações em que a distribuição espacial dos custos prejudicou significativamente o balanceamento. Entretanto, o aumento do número de processos não produziu ganhos monotônicos: o tempo médio aumentou de 0,2135 s, com nove processos, para 0,2349 s, com onze, e 0,2595 s, com doze processos, utilizando cyclic.

A estratégia block apresentou resultados competitivos com dois e três processos, alcançando *speedup* de 1,878 e 2,738, respectivamente. Entretanto, a partir de quatro processos, foram observados valores elevados de FBC, que ultrapassaram 90% em diversas configurações. Esse comportamento indica uma distribuição bastante desigual dos tempos de processamento entre os processos.

Na estratégia cyclic, o FBC permaneceu abaixo de 3% até seis processos, demonstrando uma distribuição mais equilibrada nesse intervalo. A partir de sete processos, a métrica aumentou, chegando a 32,37% com doze processos. Portanto, embora a distribuição intercalada tenha reduzido o desbalanceamento em relação à estratégia por blocos, ela não eliminou completamente a heterogeneidade do custo computacional.

A eficiência também diminuiu com o crescimento do número de processos. No melhor resultado médio do caso padrão, obtido com nove processos e cyclic, a eficiência foi de 44,19%. Isso demonstra que, apesar da redução do tempo absoluto, os recursos adicionais não produziram um ganho proporcional à quantidade de processos empregados.

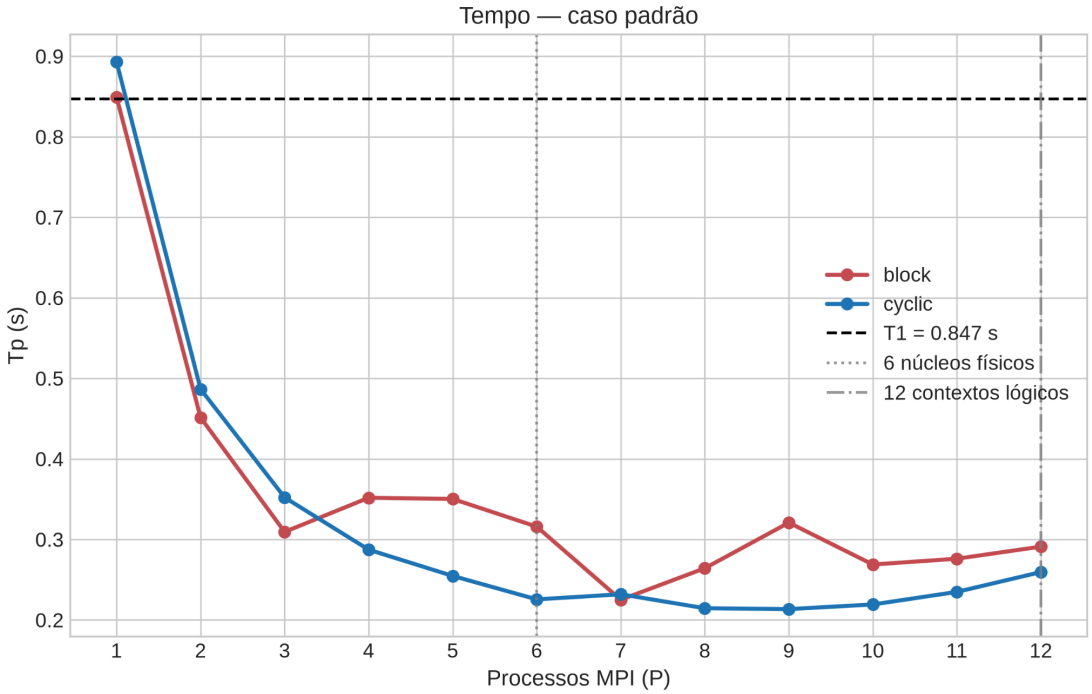


Figura 1: Tempo de execução em função do número de processos no input padrão

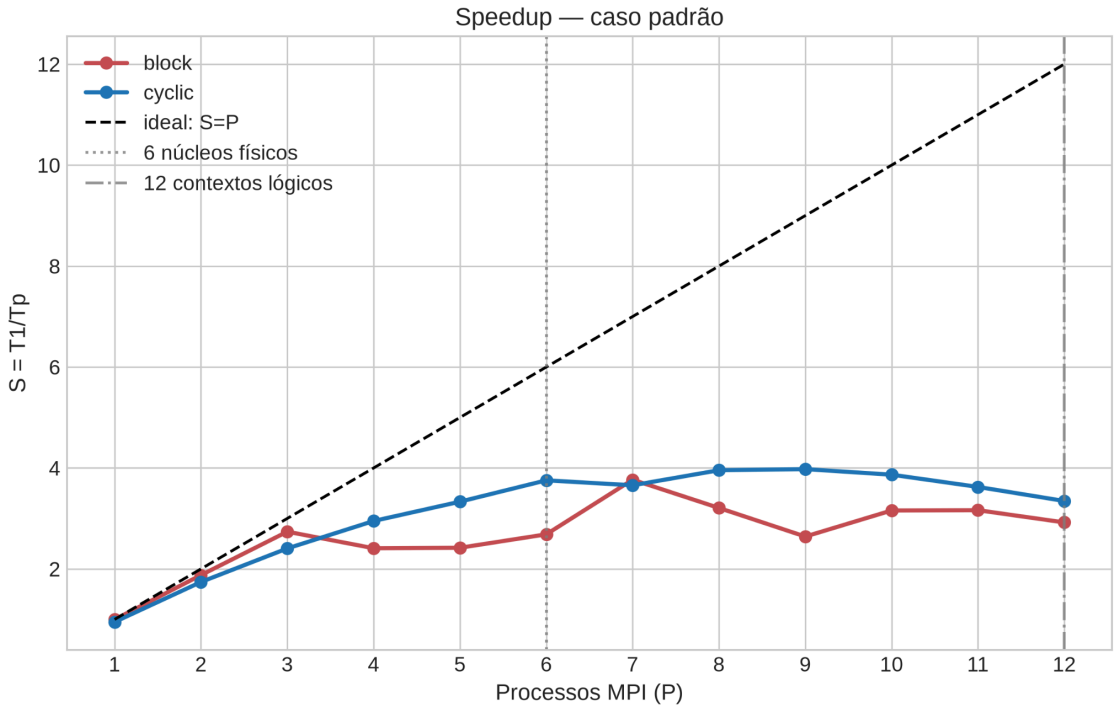


Figura 2: Speedup em função do número de processos MPI no input padrão

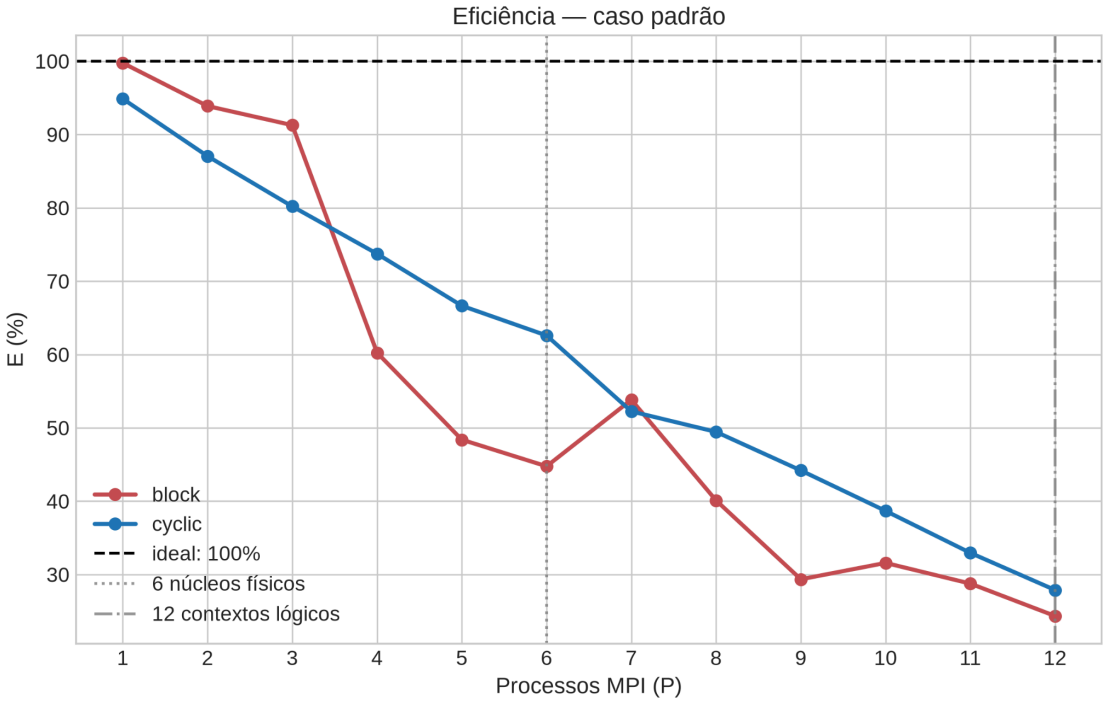


Figura 3: Eficiência paralela em função do número de processos MPI no input padrão

As Figuras 1, 2 e 3 ilustram o comportamento do tempo de execução (T_p), speedup e eficiência no cenário padrão. Na Figura 1, observa-se uma queda acentuada do tempo até 3 processos em ambas as estratégias. A partir de 4 processos, a estratégia block sofre degradação e oscilações (com picos em 4, 5 e 9 processos), enquanto a cyclic mantém redução contínua até atingir o ponto ótimo em 9 processos ($0,2135s$). Na Figura 2, nota-se um pico isolado na estratégia block com 7 processos ($S = 3,768$), decorrente de um alinhamento favorável das fronteiras dos blocos com o cardioide central da imagem. Na Figura 3, a queda constante da eficiência reflete tanto o compartilhamento de núcleos físicos via SMT (acima de 6 processos) quanto o peso relativo da comunicação coletiva em uma carga computacional leve.

3.3 Desempenho no Vale dos Cavalos-Marinhos

O segundo cenário utilizou a região do Vale dos Cavalos-Marinhos, centralizada nas coordenadas ((-0,743643887;\ 0,131825904)), com largura de (3,0x10^{-3}), resolução de (4096x4096) pixels e MAX_ITER = 5000.

Esse cenário exige maior esforço computacional por pixel e permite observar o comportamento das estratégias em uma região mais custosa do conjunto de Mandelbrot. O tempo de referência (T_1) apresentou média de **3,092420 s**.

A Tabela 3 apresenta os tempos médios de execução, os valores de *speedup* e eficiência e os fatores de balanceamento calculados para cada estratégia.

Tabela 3 — Desempenho MPI no Vale dos Cavalos-Marinhos

Processos	(T_p) block (s)	Speedup	Eficiência	FBC	(T_p) cyclic (s)	Speedup	Eficiência	FBC
1	3,1049	0,996	99,61%	0,00%	3,1424	0,984	98,41%	0,00%
2	1,8768	1,648	82,39%	33,36%	1,6137	1,916	95,82%	0,21%
3	1,2583	2,458	81,92%	29,62%	1,1090	2,789	92,95%	0,35%
4	1,0165	3,042	76,06%	48,79%	0,8623	3,586	89,66%	0,45%
5	0,8818	3,507	70,14%	64,82%	0,7184	4,306	86,11%	0,64%
6	0,7778	3,976	66,27%	65,21%	0,6153	5,027	83,78%	1,24%
7	0,6759	4,587	65,53%	63,44%	0,5764	5,367	76,67%	7,44%
8	0,6058	5,105	63,82%	63,50%	0,5271	5,870	73,37%	7,60%
9	0,5603	5,522	61,36%	63,52%	0,4860	6,372	70,80%	8,31%
10	0,5505	5,623	56,23%	66,23%	0,4758	6,507	65,07%	9,66%
11	0,5152	6,002	54,57%	64,52%	0,4318	7,166	65,15%	7,91%
12	0,5721	5,487	45,72%	71,04%	0,4471	6,925	57,71%	12,24%

No cenário do Vale dos Cavalos-Marinhos, a estratégia cyclic apresentou menor tempo médio do que block em todas as configurações com dois ou mais processos. O melhor resultado foi obtido com onze processos, alcançando **0,431841 s**, *speedup* médio de 7,166 e eficiência média de 65,15%.

A distribuição em blocos apresentou desbalanceamento expressivo. A partir de dois processos, o FBC permaneceu elevado e atingiu 71,04% com doze processos. Isso evidencia que dividir a imagem em faixas contíguas pode atribuir regiões com custos computacionais muito diferentes a processos distintos.

Por outro lado, a estratégia cyclic apresentou FBC inferior a 1,3% entre dois e seis processos. A partir de sete processos, o valor aumentou, mas permaneceu inferior ao observado em block para a mesma quantidade de processos. Essa diferença indica que a distribuição intercalada ajudou a distribuir entre os processos linhas provenientes de regiões com diferentes custos computacionais.

O aumento de processos também apresentou ganhos progressivamente menores. O tempo médio diminuiu até onze processos, mas aumentou de 0,4318 s para 0,4471 s quando a configuração passou de onze para doze processos em cyclic. Portanto, a quantidade de processos que oferece o melhor desempenho depende da combinação entre custo de processamento, balanceamento e sobrecarga da execução paralela.

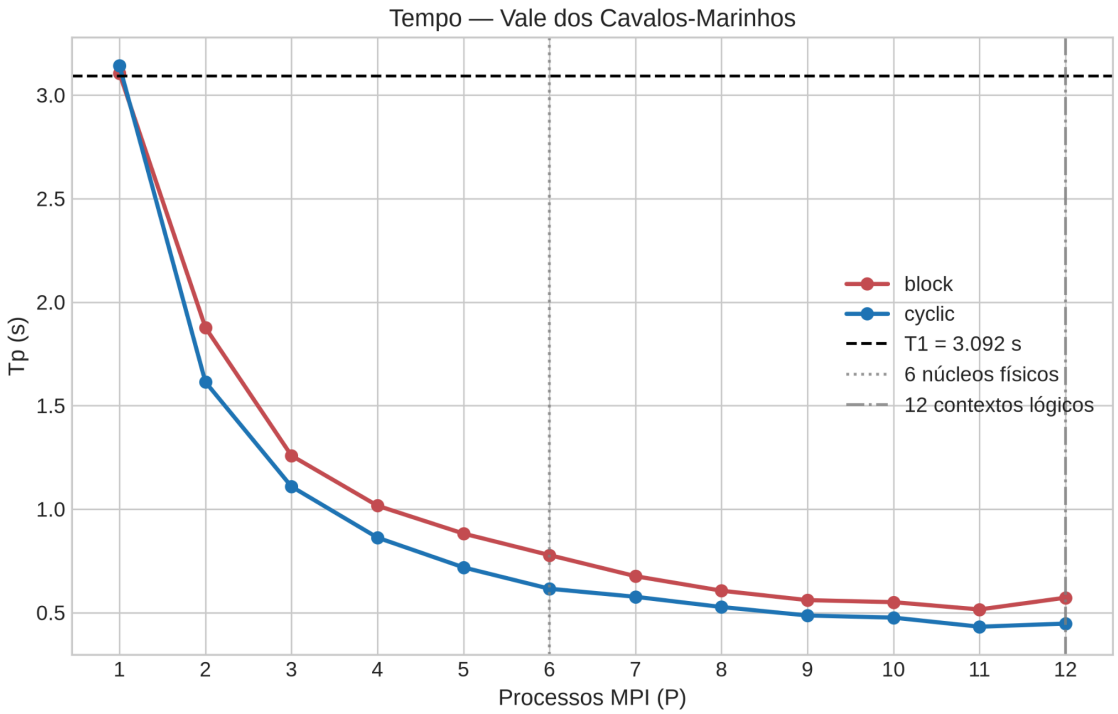


Figura 4: Tempo de execução em função do número de processos MPI no Vale dos Cavalos-Marinhos

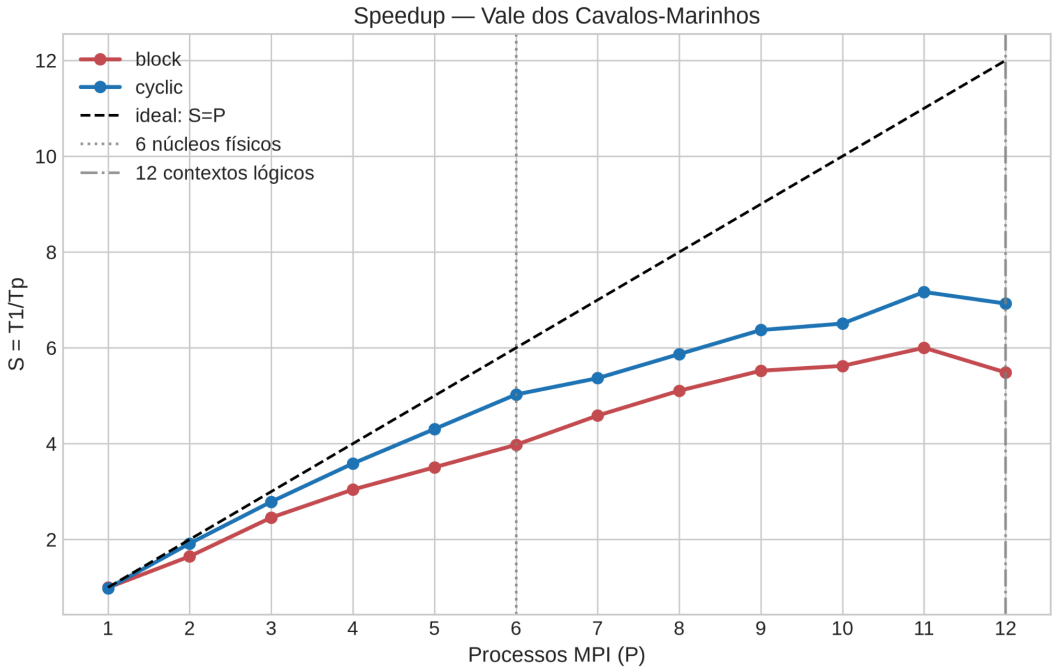


Figura 5: Speedup em função do número de processos MPI no Vale dos Cavalos-Marinhos

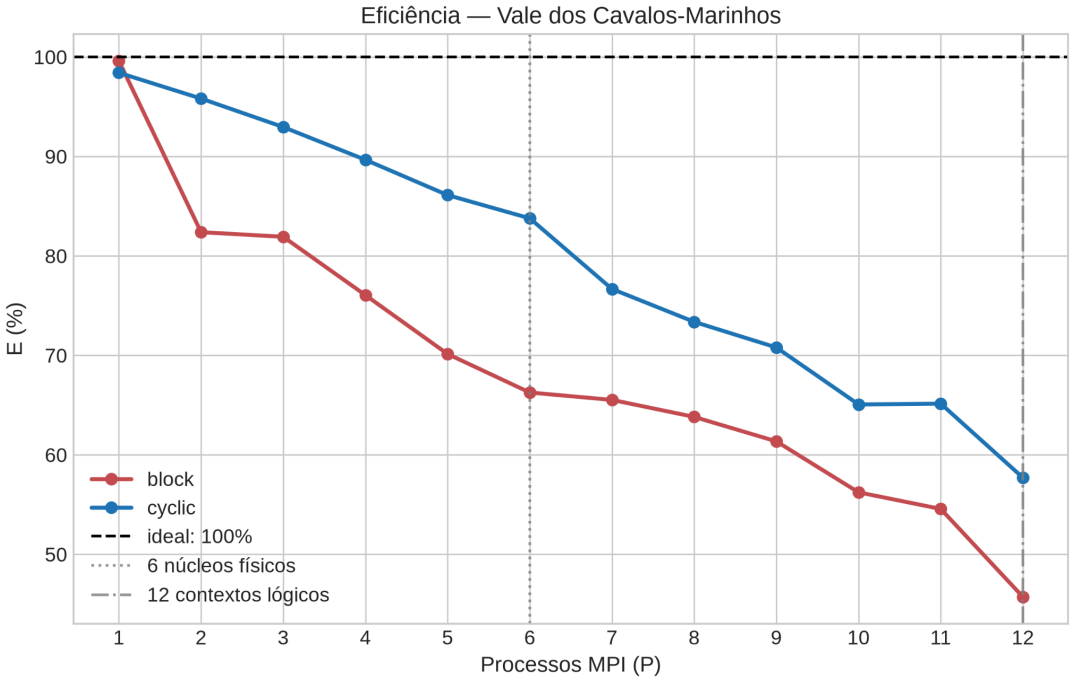


Figura 6: Eficiência em função do número de processos MPI no Vale dos Cavalos-Marinhos

As Figuras 4, 5 e 6 apresentam as curvas de desempenho para o Vale dos Cavalos-Marinhos. Diferentemente do caso padrão, a maior carga computacional ($MAX_ITER = 5000$) amortiza o custo fixo de comunicação, resultando em curvas monotônicas até 11 processos. A estratégia cyclic supera a block em todas as contagens ($P \geq 2$), sustentando eficiência de 83,78% no limite dos 6 núcleos físicos ($S = 5,027$) e atingindo speedup máximo de $7,166$ com 11 processos.

3.4 Tempos de processamento por processo e balanceamento de carga

Para complementar a análise do tempo total, foram examinados os tempos de processamento individuais registrados nos arquivos de benchmark por processo. Essa análise permite identificar diferenças que não são evidentes quando se considera apenas o tempo total da execução.

A Tabela 4 apresenta os tempos médios de processamento de cada *rank* na configuração com doze processos, considerando os dois cenários e as duas estratégias. Os valores correspondem à média das cinco execuções independentes.

Tabela 4 — Tempo médio de processamento por processo com doze processos MPI

Rank	Padrão block (s)	Padrão cyclic (s)	Seahorse block (s)	Seahorse cyclic (s)
0	0,0171	0,0883	0,2992	0,2885
1	0,0213	0,1026	0,3148	0,2922
2	0,1064	0,0836	0,4789	0,2898
3	0,2083	0,0832	0,4369	0,2956
4	0,0790	0,0925	0,3518	0,2983
5	0,0924	0,0855	0,3811	0,2857
6	0,0966	0,0933	0,2906	0,2934
7	0,0822	0,0922	0,1583	0,2985
8	0,2145	0,0834	0,1433	0,2990

9	0,1142	0,0890	0,1617	0,2936
10	0,0208	0,0844	0,2693	0,2879
11	0,0284	0,0998	0,4315	0,2909

No caso padrão, a distribuição block produziu diferenças acentuadas entre os tempos dos processos. Enquanto os *ranks* 0, 1, 10 e 11 apresentaram tempos médios inferiores a 0,03 s, os *ranks* 3 e 8 ultrapassaram 0,20 s. Essa diferença está associada à distribuição espacial desigual do custo de processamento: algumas faixas horizontais exigem muito mais iterações do que outras.

Com cyclic, os tempos médios individuais ficaram aproximadamente entre 0,083 s e 0,103 s, indicando uma distribuição mais uniforme do trabalho no cenário padrão. No Vale dos Cavalos-Marinhos, a estratégia block também apresentou diferenças expressivas, com tempos médios entre 0,143 s e 0,479 s. Na estratégia cyclic, os tempos ficaram aproximadamente entre 0,286 s e 0,299 s.

Essas observações são coerentes com os valores de FBC apresentados nas Tabelas 2 e 3. No caso padrão com doze processos, o FBC médio de block foi de 92,29%, enquanto o de cyclic foi de 32,37%. No cenário Seahorse, os valores correspondentes foram 71,04% e 12,24%.

Embora o esquema cíclico também apresenta diferenças entre processos, seus resultados indicam que a distribuição intercalada reduz a concentração de trabalho computacionalmente custoso em determinados *ranks*. Essa característica explica parte da melhoria de desempenho observada em relação à distribuição por blocos, especialmente no cenário do Vale dos Cavalos-Marinhos.

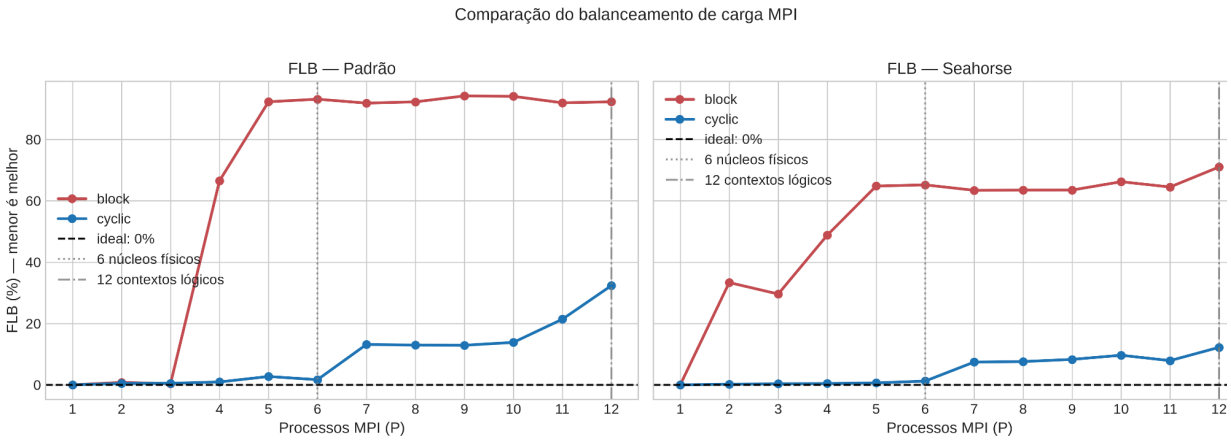


Figura 7: Comparação do Fator de Balanceamento de Carga (FBC) entre o caso padrão e o Vale dos Cavalos-Marinhos

A Figura 7 compara a evolução do FBC nos dois cenários para ambas as estratégias. Na estratégia block, o FBC salta abruptamente a partir de 4 processos no caso padrão (estabilizando acima de 90%) e a partir de 2 processos no cenário Seahorse (atingindo 71,04% em 12 processos). Em contrapartida, a distribuição cyclic mantém o FBC próximo de 0% até 6 processos nos dois cenários, apresentando aumento moderado apenas na região de SMT ($P > 6$) devido à contenção de hardware entre processos que compartilham o mesmo núcleo físico.

3.5 Custo de comunicação e rearranjo dos resultados

Além do cálculo dos pixels, a implementação MPI necessita reunir os resultados parciais no processo raiz. Na estratégia block, essa operação é realizada com MPI_Gather ou MPI_Gatherv, dependendo de a quantidade de linhas ser divisível pelo número de processos. Na estratégia cyclic, os resultados são recebidos por meio de MPI_Gatherv em um buffer auxiliar, sendo posteriormente reorganizados para recuperar a ordem original das linhas.

A Tabela 5 apresenta o custo de comunicação nas configurações de melhor tempo médio de cada estratégia e nas configurações com doze processos.

Tabela 5 — Custo de coleta e rearranjo nas configurações selecionadas

Cenário	Estratégia	Processos	Coleta (ms)	Rearranjo (ms)	Custo medido total (ms)	Parcela de (T_p)

Padrão	block	7	39,4	0,0	39,4	17,5%
Padrão	cyclic	9	50,6	56,3	106,9	50,1%
Padrão	block	12	63,0	0,0	63,0	21,6%
Padrão	cyclic	12	63,8	63,5	127,3	49,0%
Seahorse	block	11	57,1	0,0	57,1	11,1%
Seahorse	cyclic	11	57,1	58,7	115,8	26,8%
Seahorse	block	12	67,7	0,0	67,7	11,8%
Seahorse	cyclic	12	60,6	63,4	124,0	27,7%

Os resultados mostram que a estratégia cyclic acrescenta uma etapa de rearranjo que não existe na distribuição por blocos. Esse custo ficou entre aproximadamente 41 ms e 63 ms nas configurações avaliadas. Por sua vez, o tempo de coleta variou de aproximadamente 36 ms a 68 ms, dependendo do cenário e da quantidade de processos.

No caso padrão, a comunicação e o rearranjo representaram aproximadamente 50,1% do tempo total na configuração cyclic com nove processos, que apresentou o menor tempo médio. Com doze processos, essa parcela permaneceu próxima de 49,0%. Isso demonstra que, para esse cenário, as operações de reunião e reorganização dos resultados constituem uma parcela significativa do tempo de execução.

No cenário do Vale dos Cavalos-Marinhos, a parcela correspondente foi menor em termos relativos. Com onze processos e cyclic, o custo medido de comunicação e rearranjo foi de aproximadamente 26,8% do tempo total. Como esse cenário exige mais processamento por pixel, o custo de comunicação representa uma fração menor da execução, embora continue relevante.

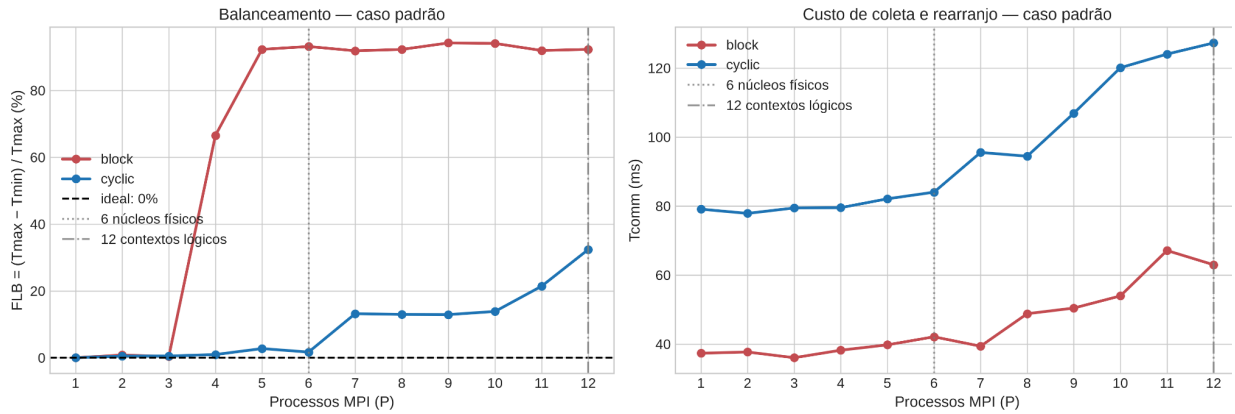


Figura 8: Relação entre o Fator de Balanceamento de Carga e o custo de coleta/rearranjo no caso padrão

A Figura 8 sintetiza o *trade-off* central da implementação no caso padrão. Enquanto o gráfico à esquerda mostra que a estratégia cyclic corrige o severo desbalanceamento de carga da estratégia block, o gráfico à direita evidencia o preço dessa escolha: o custo combinado de coleta e rearranjo em memória (memcpy) na distribuição cyclic parte de ~80 ms e atinge $127,3ms$ com 12 processos, mais que o dobro do custo de coleta direta da estratégia block ($63,0ms$).

3.6 Comparação entre as implementações MPI e OpenMP

A comparação entre MPI e OpenMP permite analisar como dois modelos de programação paralela se comportam na geração do Conjunto de Mandelbrot. Para tornar os experimentos mais comparáveis, as duas implementações paralelas foram executadas sem a otimização de simetria, processando todas as linhas da imagem. Os testes consideram o input padrão, com resolução de (4096x4096) pixels e MAX_ITER = 1000.

A Tabela 6 apresenta as melhores configurações de cada estratégia MPI e o resultado do OpenMP com dynamic, utilizando chunk = 1.

Tabela 6 — Comparação de desempenho entre OpenMP e MPI no input padrão

Implementação	Configuração	Tempo médio (s)	Speedup médio	Eficiência média
MPI — block	7 processos	0,225105	3,768	53,82%

MPI — cyclic	9 processos	0,213505	3,977	44,19%
OpenMP — dynamic	11 threads	0,106381	8,149	74,08%
OpenMP — dynamic	12 threads	0,103241	8,077	67,31%

Nota: as configurações do OpenMP com 11 e 12 threads são apresentadas separadamente porque a primeira obteve o maior speedup médio, enquanto a segunda apresentou o menor tempo médio. Os speedups e as eficiências utilizam as respectivas referências de cada implementação.

Os resultados mostram que a versão OpenMP apresentou menor tempo de execução do que as duas estratégias MPI nas melhores configurações observadas. Enquanto a distribuição cyclic atingiu tempo médio de 0,213505 s com nove processos, o OpenMP alcançou 0,103241 s com doze threads. Considerando os tempos medidos, a execução OpenMP foi aproximadamente 51,7% menor.

O gráfico de speedup também evidencia diferenças no aproveitamento dos recursos. O OpenMP alcançou speedup médio de 8,149 com onze threads, enquanto o melhor resultado MPI foi de 3,977, obtido com nove processos e a estratégia cyclic. Esse comportamento indica que, neste ambiente experimental, a implementação com memória compartilhada obteve ganhos relativos maiores em relação à sua própria referência.

Entretanto, esses valores devem ser interpretados considerando as características de cada modelo. A implementação MPI precisa reunir os resultados produzidos pelos diferentes processos e, na distribuição cíclica, reorganizar as linhas no processo raiz. Essas operações acrescentam custos que não existem da mesma forma na implementação OpenMP. Além disso, os tempos de referência usados no cálculo das métricas são diferentes: o MPI utiliza sua referência de execução com um processo, enquanto o OpenMP utiliza a versão sequencial da Etapa 1.

Portanto, a comparação evidencia uma vantagem de desempenho do OpenMP nas configurações avaliadas, mas não permite atribuir toda a diferença exclusivamente ao modelo de programação. Os resultados também refletem as operações adicionais da implementação MPI e as particularidades das medições realizadas.

A Figura 9 ilustra o distanciamento entre os dois modelos de programação à medida que o paralelismo escala. Até 3 unidades de processamento, o desempenho de ambos é similar. A partir de 4 processos/threads, o OpenMP mantém crescimento quase linear até 6 núcleos físicos ($E \approx 95\%$) e atinge speedup de $8,149$ com 11 threads, enquanto o MPI estagna próximo a $S \approx 4,0$ devido ao gargalo de comunicação coletiva e cópia de buffers no processo raiz.

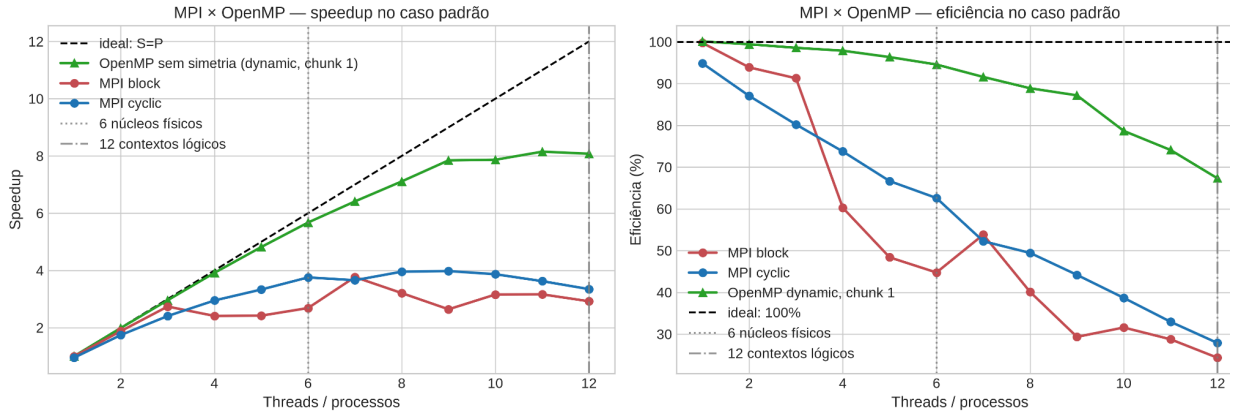


Figura 9: Comparação de speedup e eficiência entre MPI (block e cyclic) e OpenMP (dynamic, chunk=1) sem simetria no caso padrão

4. Discussão

4.1 Análise da Corretude

A verificação de corretude comprovou que tanto a estratégia block quanto a cyclic em MPI reproduzem bit a bit a saída da implementação sequencial de referência. Nos dois cenários avaliados ($16.777.216$ pixels por imagem), houve 0% de divergência e coincidência exata dos hashes SHA-256. Isso confirma que a decomposição estática de domínio, o envio coletivo via MPI_Gather/MPI_Gatherv e a rotina de reconstrução de índices via memcpy no modo cyclic preservam a integridade numérica do algoritmo.

4.2 Comparação entre a versão sequencial e paralela em MPI

A comparação entre a versão sequencial e as versões paralelas em MPI foi realizada nos modos padrão e Seahorse, utilizando as estratégias de distribuição de linhas block e cyclic, com testes de 1 a 12 processos e cinco repetições para cada configuração. Os

resultados foram avaliados por meio do tempo de execução, do speedup e da eficiência paralela.

No modo padrão, a referência (T_1) apresentou tempo médio de 0,846998 s, enquanto a melhor configuração paralela alcançou um speedup médio de 3,977 vezes, utilizando 9 processos com a estratégia cyclic. Nessa configuração, o tempo médio de execução foi de 0,213505 s, com eficiência de 44,19%.

No modo Seahorse, a referência (T_1) apresentou tempo médio de 3,092420 s. A melhor configuração paralela obteve um speedup médio de 7,166 vezes, utilizando 11 processos com a estratégia cyclic, com tempo médio de execução de 0,431841 s e eficiência de 65,15%.

Em relação às estratégias de distribuição, a block apresentou melhores resultados em diversas configurações com poucos processos, enquanto a cyclic geralmente obteve tempos menores a partir de quantidades intermediárias de processos, embora tenham ocorrido algumas exceções. Também foi observado que aumentar o número de processos não garantiu uma redução contínua do tempo de execução, devido aos custos de comunicação, sincronização e reorganização dos dados.

Comparando os dois modos, o Seahorse alcançou maior speedup e eficiência em sua melhor configuração. Entretanto, isso não significa que tenha apresentado menor tempo absoluto de execução, pois sua versão sequencial também exigiu mais tempo de processamento.

Assim, os resultados demonstram que o MPI pode reduzir significativamente o tempo de execução do conjunto de Mandelbrot. Entretanto, o desempenho depende da carga de trabalho, do número de processos e da estratégia de distribuição utilizada. Portanto, a escolha da configuração mais adequada deve considerar não apenas a quantidade de processos, mas também os custos adicionais envolvidos na execução paralela.

4.3 Comparação entre as estratégias de distribuição: block vs. cyclic

A comparação direta evidencia um compromisso claro entre balanceamento de carga e sobrecarga de gerenciamento de dados:

- **Balanceamento de carga (FBC) e Desempenho:** A estratégia block é inadequada para o Conjunto de Mandelbrot quando $P \geq 4$, pois agrupa faixas horizontais contíguas. Como os pontos pertencentes ao conjunto (MAX_ITER) concentram-se na faixa central da imagem, processos específicos (como os

ranks 3 e 8 com 12 processos no caso padrão) levam mais de $0,20s$, enquanto processos nas bordas (ranks 0, 1, 10 e 11) terminam em menos de $0,03s$, gerando FBC superior a 90%. A estratégia cyclic distribui linhas adjacentes entre processos distintos (passo P), garantindo que todos os ranks amostrem uniformemente regiões leves e pesadas. Isso reduz o FBC para menos de 3% até 6 processos no caso padrão e menos de 1,3% no Seahorse.

- **Custo de comunicação e Complexidade:** A estratégia block é mais simples de implementar e permite que o MPI_Gather/MPI_Gatherv escreva os dados recebidos diretamente na posição final da matriz no rank 0. Já a estratégia cyclic exige um buffer auxiliar na coleta seguido de um laço de reorganização (memcpy linha a linha) no processo raiz. No caso padrão com 12 processos, esse rearranjo custou $63,5ms$, dobrando o overhead pós-cálculo em relação ao block ($127,3ms$ contra $63,0ms$).
- **Escolha mais adequada:** Apesar do custo adicional de reorganização em memória, a estratégia **cyclic** mostrou-se superior para o problema, vencendo em todas as configurações com $P \geq 2$ no cenário Seahorse e na maioria das configurações com $P \geq 4$ no caso padrão, pois o ganho obtido ao eliminar a ociosidade dos processos supera a penalidade do rearranjo local.

4.4 Impacto do número de processos e relação com as Leis de Amdahl e Gustafson

O comportamento das curvas de speedup reflete diretamente os conceitos das Leis de Amdahl (escalabilidade forte) e de Gustafson (escalabilidade ligada ao aumento de carga):

- **Lei de Amdahl no Caso Padrão:** Mantendo o problema fixo (4096×4096, $MAX_ITER = 1000$), a fração efetivamente serial e não paralelizada da execução — composta pela coleta coletiva (MPI_Gatherv), pela reorganização sequencial das linhas no processo mestre (memcpy) e pela sincronização — passa a dominar o tempo total (T_p). Como na configuração de 9 processos (cyclic) a comunicação e o rearranjo consomem 50,1% de T_p ($106,9ms$ de $213,5ms$), o speedup máximo fica severamente limitado pelo teto de Amdahl, estagnando próximo a $3,98\times$. Além disso, acima de 6 processos, os processos passam a disputar recursos dos mesmos núcleos físicos via SMT, degradando a eficiência para 27,88% em 12 processos.

- **Relação com a Lei de Gustafson no Cenário Seahorse:** Quando o trabalho computacional aumenta em 5 vezes no limite de iterações ($MAX_ITER = 5000$), o tempo gasto no cálculo puro cresce significativamente, enquanto o volume de dados trafegados na rede/memória permanece idêntico (4096×4096 inteiros, ~64 MB). Com isso, a fração de comunicação e rearranjo cai de ~50% para 26,8% do tempo total (em 11 processos). Essa diluição da fração sequencial/comunicacional permite que o sistema escale melhor com o aumento de recursos, elevando o speedup máximo de $3,977\ \times$ para $7,166\ \times$.

4.5 Gargalos encontrados e limitações da implementação

Sobrecarga de comunicação e serialização no Mestre: O uso de MPI_Gatherv transfere ~64 MB de dados para o rank 0 ao final do cálculo. No modo cyclic, apenas o rank 0 executa a reorganização das 4096 linhas em memória enquanto os outros $P - 1$ processos permanecem ociosos até o término da medição, criando um gargalo sequencial de ~60 ms.

Ausência de balanceamento dinâmico: Ambas as abordagens implementadas (block e cyclic) são estáticas. Embora o cyclic mitigue o desbalanceamento geométrico, sob condições de SMT ($P > 6$), variações de escalonamento do sistema operacional elevaram o FBC do cyclic para até 32,37% no caso padrão, pois processos atrasados não podem repassar suas linhas pendentes a processos livres.

Limitação de Hardware e Escopo Experimental: Os testes foram executados em uma única máquina com 6 núcleos físicos e 12 threads lógicas, utilizando memória compartilhada como transporte subjacente da biblioteca MPI. Além disso, a análise focou na escalabilidade forte (resolução fixa em 4096×4096); testes adicionais variando a resolução proporcionalmente ao número de processos seriam necessários para uma avaliação isolada de escalabilidade fraca em memória distribuída.

4.6 Comparação com a solução OpenMP

No ambiente de nó único avaliado, a implementação em memória compartilhada (OpenMP com schedule(dynamic, 1)) superou amplamente a versão em memória distribuída (MPI), atingindo tempo médio 51,7% menor ($0,1032s$ contra $0,2135s$) e o dobro de speedup ($8,149\ \times$ contra $3,977\ \times$) no caso padrão sem simetria. As causas dessa diferença são estruturais:

- **Espaço de endereçamento único vs. Troca de mensagens:** No OpenMP, todas as threads escrevem diretamente na matriz global alocada na memória

principal, resultando em custo zero de coleta e rearranjo. No MPI, cada processo possui espaço de endereçamento isolado, exigindo cópia explícita via MPI_Gatherv e memcpy (que consumiu $106,9ms$ na melhor configuração MPI, praticamente todo o tempo gasto pela execução inteira do OpenMP).

- **Escalonamento Dinâmico Nativo vs. Decomposição Estática:** O OpenMP oferece distribuição dinâmica em nível de linha (dynamic, 1) com custo mínimo de sincronização via variáveis atômicas/locks internos, adaptando-se em tempo real ao SMT. No MPI, implementar um esquema dinâmico exigiria comunicação ponto a ponto contínua (MPI_Send/MPI_Recv), enquanto a abordagem estática cyclic sofre quando há contenção de núcleos.
- **Vantagem do MPI:** Apesar do menor desempenho em máquina única, o código MPI desenvolvido possui a vantagem da escalabilidade horizontal: pode ser executado sem alterações em múltiplos nós de um cluster computacional, superando o limite físico de núcleos de uma única placa-mãe ao qual o OpenMP está restrito.

4.7 Sugestões de melhorias e otimizações futuras

Esquema Mestre-Trabalhador Dinâmico: Implementar distribuição dinâmica com MPI_Send e MPI_Recv (utilizando tags de requisição de trabalho), na qual o processo mestre envia blocos pequenos de linhas sob demanda aos processos trabalhadores, eliminando o FBC residual na região de SMT ($P > 6$).

Uso de Tipos Derivados (MPI_Type_vector) ou Paralelização do Rearranjo: Evitar a cópia manual sequencial com memcpy no modo cyclic definindo um tipo derivado com *stride* no MPI para que o próprio MPI_Gatherv posicione as linhas intercaladas diretamente na matriz final, ou paralelizar a etapa de rearranjo no nó mestre.

Programação Híbrida (MPI + OpenMP): Utilizar 1 processo MPI por nó físico (eliminando cópias de memória intra-nó) combinado com threads OpenMP (dynamic, 1) dentro de cada nó para explorar os núcleos locais com balanceamento dinâmico e sem custo de rearranjo.

5. Conclusão

5.1 Principais Aprendizados

O desenvolvimento da Etapa 2 permitiu compreender como a programação paralela com MPI pode ser utilizada para reduzir o tempo de execução da geração do Conjunto de Mandelbrot em um ambiente de memória distribuída. A implementação possibilitou avaliar diferentes quantidades de processos e comparar as estratégias de distribuição `block` e `cyclic`, considerando o tempo de execução, o *speedup* e a eficiência paralela.

Outro aprendizado importante foi compreender o desafio de distribuir uma carga de trabalho heterogênea. Embora o cálculo de cada pixel seja independente, o número de iterações necessárias varia conforme a região da imagem. Por esse motivo, alguns processos podem receber tarefas mais demoradas e terminar seus cálculos depois dos demais. A comparação entre `block` e `cyclic` permitiu observar como a organização da distribuição influencia o balanceamento de carga e o desempenho geral da aplicação.

5.2 Considerações Finais

Os resultados mostraram que a paralelização proporcionou ganhos de desempenho nos modos padrão e Seahorse. No modo padrão, o melhor speedup médio observado foi de 3,977 vezes (0,213505 s), com 9 processos e a estratégia cyclic. No modo Seahorse, o melhor resultado médio foi de 7,166 vezes (0,431841 s), com 11 processos e a mesma estratégia. Esses resultados demonstram que a distribuição do trabalho entre processos pode reduzir significativamente o tempo de processamento, embora os ganhos não aumentem necessariamente de forma proporcional ao número de processos utilizados.

A utilização do MPI também evidenciou que o processamento paralelo envolve custos adicionais, como comunicação entre processos, sincronização e reunião dos resultados. Esses custos podem limitar os ganhos obtidos com o aumento do número de processos e explicam por que algumas configurações apresentam desempenho inferior ao de outras com menos processos.

Conclui-se que a estratégia cyclic apresentou os melhores resultados individuais observados nos dois modos avaliados. Entretanto, o desempenho depende da quantidade de processos, das características da carga de trabalho e dos custos de comunicação e reorganização. Dessa forma, o projeto demonstrou que obter bons resultados com programação paralela exige não apenas dividir o processamento, mas também buscar uma distribuição equilibrada das tarefas e avaliar cuidadosamente os custos adicionais da execução.

6. Referências

ETVORELLANA. **RepoDEC107-PP-2026_2**. GitHub. Disponível em: https://github.com/etvorellana/RepoDEC107-PP-2026_2. Acesso em: 28 out. 2026.

MESSAGE PASSING INTERFACE FORUM. **MPI: A Message-Passing Interface Standard Version 4.1**. 2023. Disponível em: https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf. Acesso em: 28 out. 2026.

MPICH. **MPICH v4.3.2 Documentation / User's Guide**. 2025. Disponível em: https://www.mpich.org/documentation/guides/. Acesso em: 28 out. 2026.

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, Enzo Fernandes Sarmento e Jéssica Meira Costa, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado "Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time)", submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

( ) Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(x) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável): As ferramentas utilizadas foram: Codex, Antigravity CLI, Gemini. O uso se deu exclusivamente nas seguintes etapas do projeto:

- Revisão gramatical do relatório
- Geração de scripts base para testes de benchmark
- Geração de scripts python para plotagem dos gráficos de acordos com os dados extraídos
- Estruturação inicial de ideias e revisão bibliográfica

Termo de Responsabilidade: Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmo que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA

foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus - BA, 28 de outubro de 2026

Enzo Fernandes Sarmento - Matrícula: 202410024
Jéssica Meira Costa - Matrícula: 202410513